Protecting hospital cloud systems when the protectors themselves can be attacked

A plain-English guide to the Distributed Cyber-Resilience Platform, for a presentation to a non-expert audience

What to remember in one minute. A clinic's patient portal, login service, health-record service and database run in the cloud. If one of them is taken over, someone must cut it off quickly, replace it with a clean copy, check the copy, and bring it back carefully. Normally one central program decides. If that program is fooled, hacked or stops, the whole clinic is exposed. This project replaces the single decision-maker with **four identical security agents** that must **agree, three out of four,** before anything is cut off. One liar or one failure cannot cause harm or stop protection.

Honest status. The agents, the signatures, the 3-of-4 agreement and the recovery pipeline are real code and were tested many times against a **simulator**. **Nothing has yet been verified on a real cluster**; the live-verified table in section 6 is empty on purpose and is for the presenter to fill with their own runs. All patient data is **synthetic** (made up).

1. The problem, in plain English

Healthcare organisations run applications in the cloud: a portal where patients and staff look at records, a service that checks who may log in, a service that serves the health records, and a database that stores them. Attackers go after exactly these. When one is compromised, three things matter:

- **Speed**: cut the compromised system off before it leaks or spreads.
- **Safety**: never cut off a healthy system by mistake, because clinicians then cannot see records.
- **Care in coming back**: replace it with a clean copy, prove the copy is clean, and give access back step by step, not all at once.

The usual design has one central controller making these decisions. That controller becomes the single weak point: if it is tricked into accusing a healthy service, it cuts it off; if it crashes, nothing is protected. In the simulator, a lying central controller cut off a healthy service **every time (20 of 20 trials)**, and with the controller down a real attack was **never** contained (0 of 20).

2. What I built

Four workloads, each with its own security agent sitting next to it (agents A to D). The agents are **identical peers**: no agent is the boss.

Agent	Protects	What it does for patients and staff	If it is isolated
A	Patient Portal	shows patient records	no other service depends on it; the portal itself is unavailable
B	Login & Access Service	signs people in and checks their access	the Patient Portal and the EHR API cannot work
C	Electronic Health Records (EHR) API	serves patient records	the Patient Portal cannot show patient records
D	Clinical Database	stores patient records	the EHR API cannot serve records, so the portal cannot show them

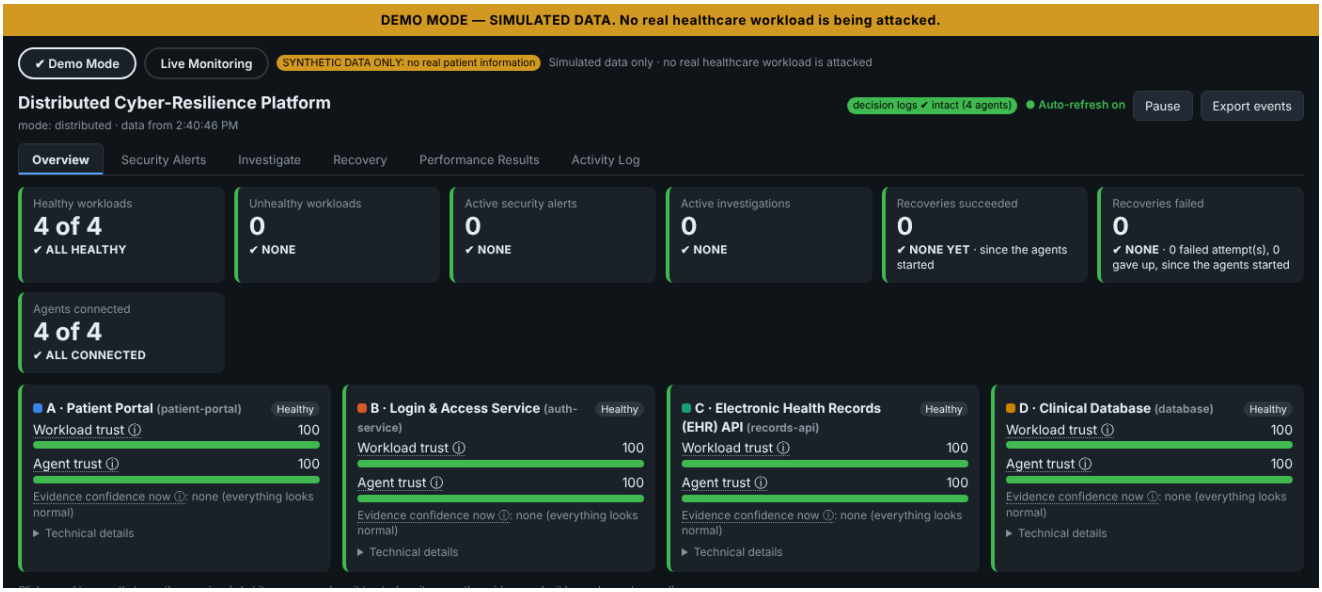
The dashboard shows this same table, and works out the last column from which service calls which (checked against the application code), so nothing is typed in by hand. The suggested recovery order shown beside it is only a suggestion for a human operator; it does not change what the agents do, and there are no invented clinical urgency scores.

How a decision is made

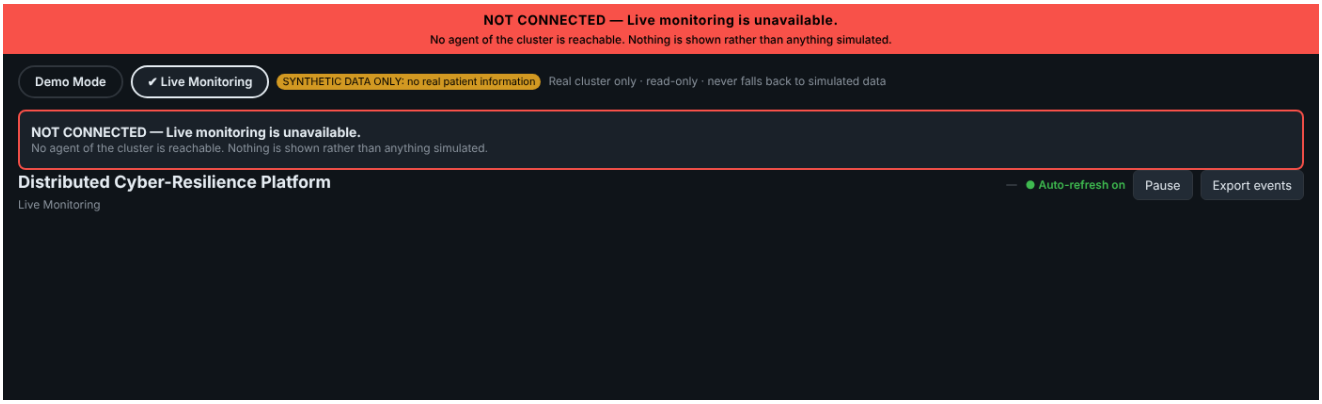
1. **See.** Each agent watches its workload and its neighbours: *Network Activity* (unexpected connections), *Running Processes* (programs that should not be there), *File Changes* (files that differ from the known-good copy) and *Failed Logins*.
2. **Share evidence.** Agents send each other signed evidence. A forged or tampered message is rejected and the sender loses trust.
3. **Agree (3 of 4).** Isolating a workload needs at least three agents to sign a vote. A missing vote is never counted as approval.
4. **Isolation.** The workload is cut off from the network (a Kubernetes network policy), and a read-only copy of the evidence is saved first.
5. **Replace and check.** A clean copy is started from the known-good image. The **Recovery Check** tests that it is healthy, that its files match, that it behaves normally and that it is really new.
6. **Return in steps.** Limited Access, Monitored Access, Confirmed by Peers, then **Normal Access Restored**, each step only when the evidence supports it. If recovery keeps failing, the workload stays isolated and a person is asked to step in.

3. Two pages that are never mixed: Demo Mode and Live Monitoring

The dashboard has a toggle at the top. **Demo Mode** shows the simulator and has the scenario buttons. **Live Monitoring** shows only the real cluster and has no buttons at all. The two pages share no data, and the server (not just the page) refuses to show simulated data on the live page. If the cluster cannot be reached the live page says so instead of showing anything else.



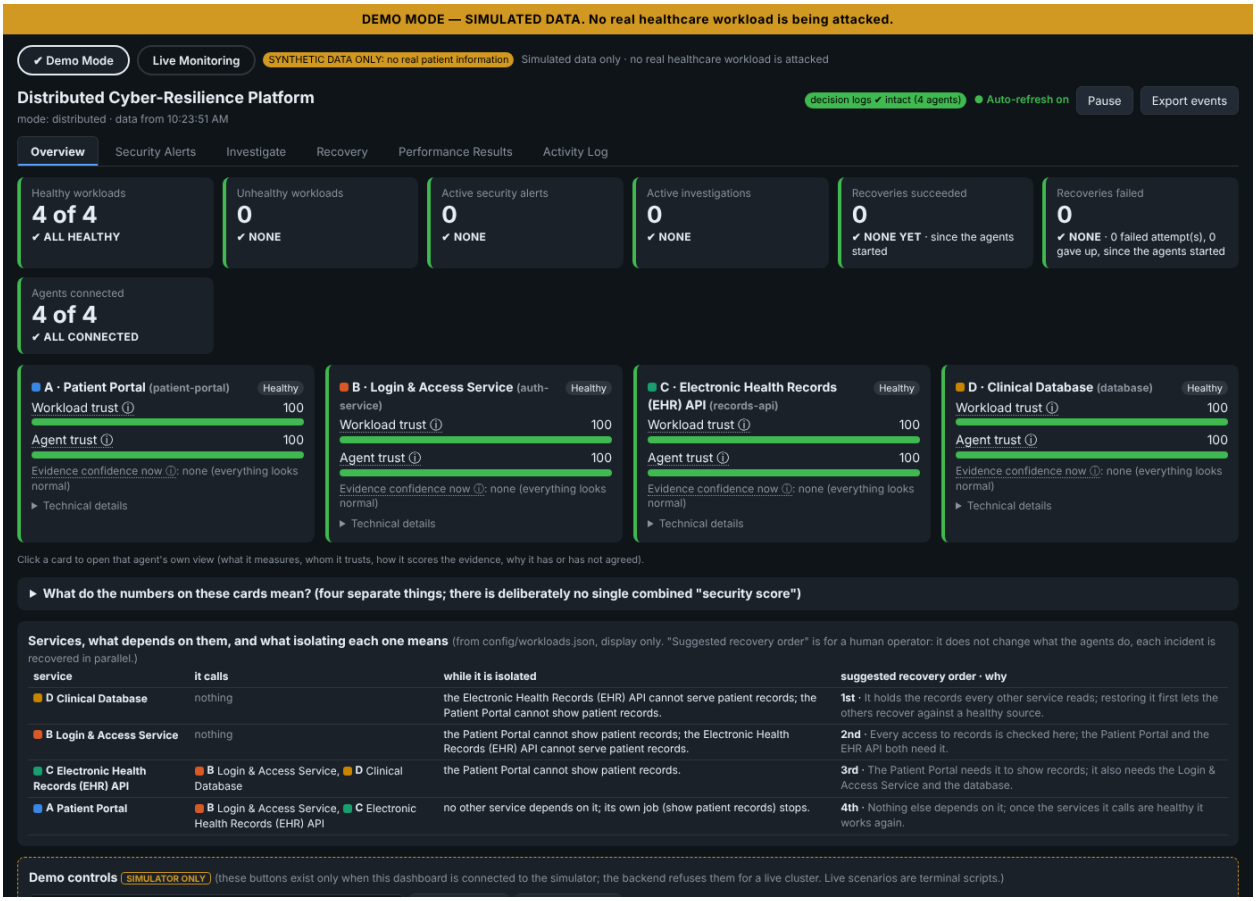
Demo Mode (simulator). Amber banner: simulated data, no real healthcare workload is attacked. The toggle, the 'synthetic data only' label, the counters and the four workload cards.



Live Monitoring with no cluster reachable (real behaviour of the current build): it says NOT CONNECTED and shows nothing from the simulator. When connected it shows a green banner with the cluster name, how many of the four agents answer and how fresh the data is.

Placeholder: Live Monitoring, connected. A screenshot of the connected live page can only be taken on the real cluster. Presenter: paste your own screenshot here after your run (see docs/LIVE_DEMO.md). None is included because none has been taken.

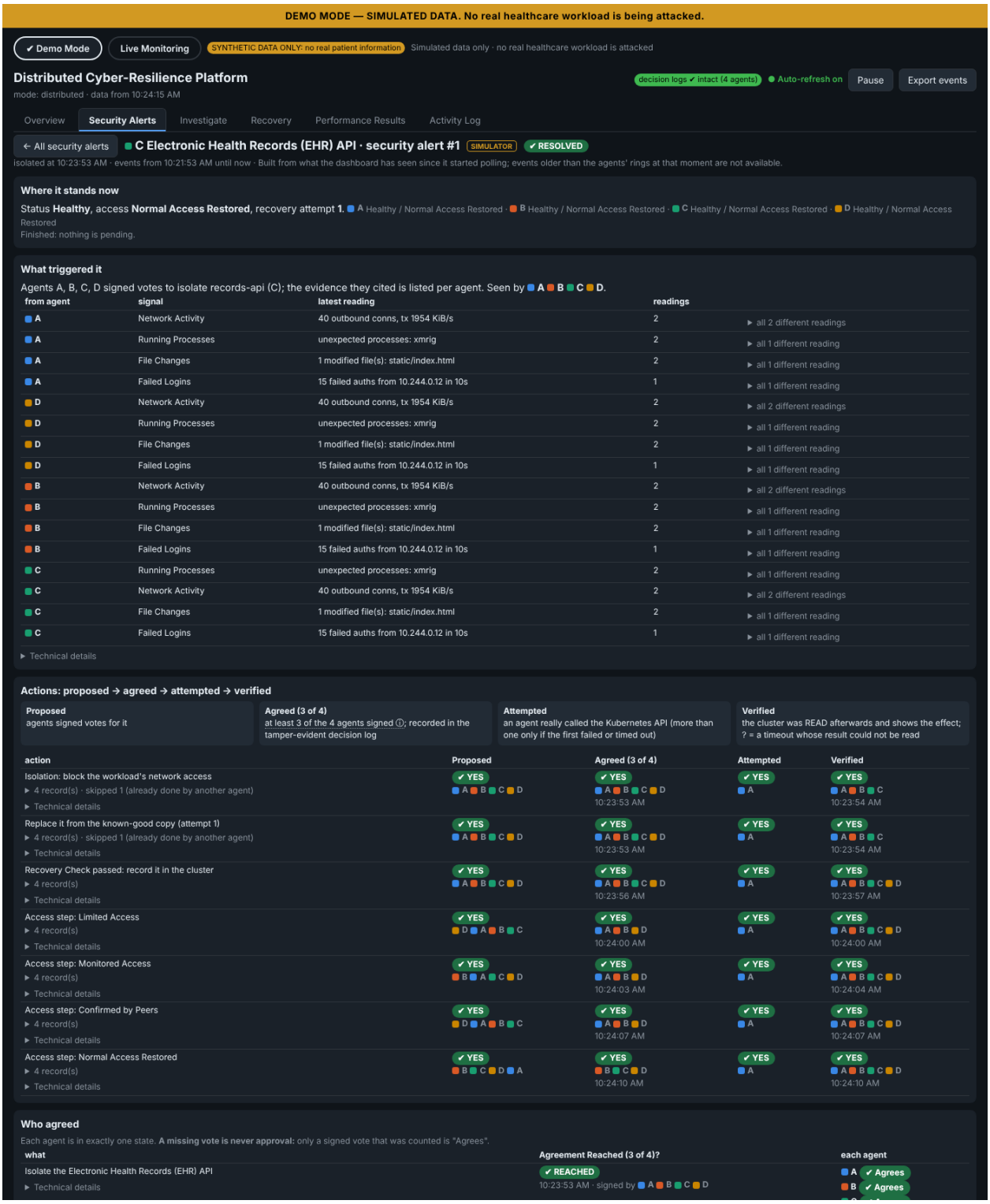
4. Reading the dashboard



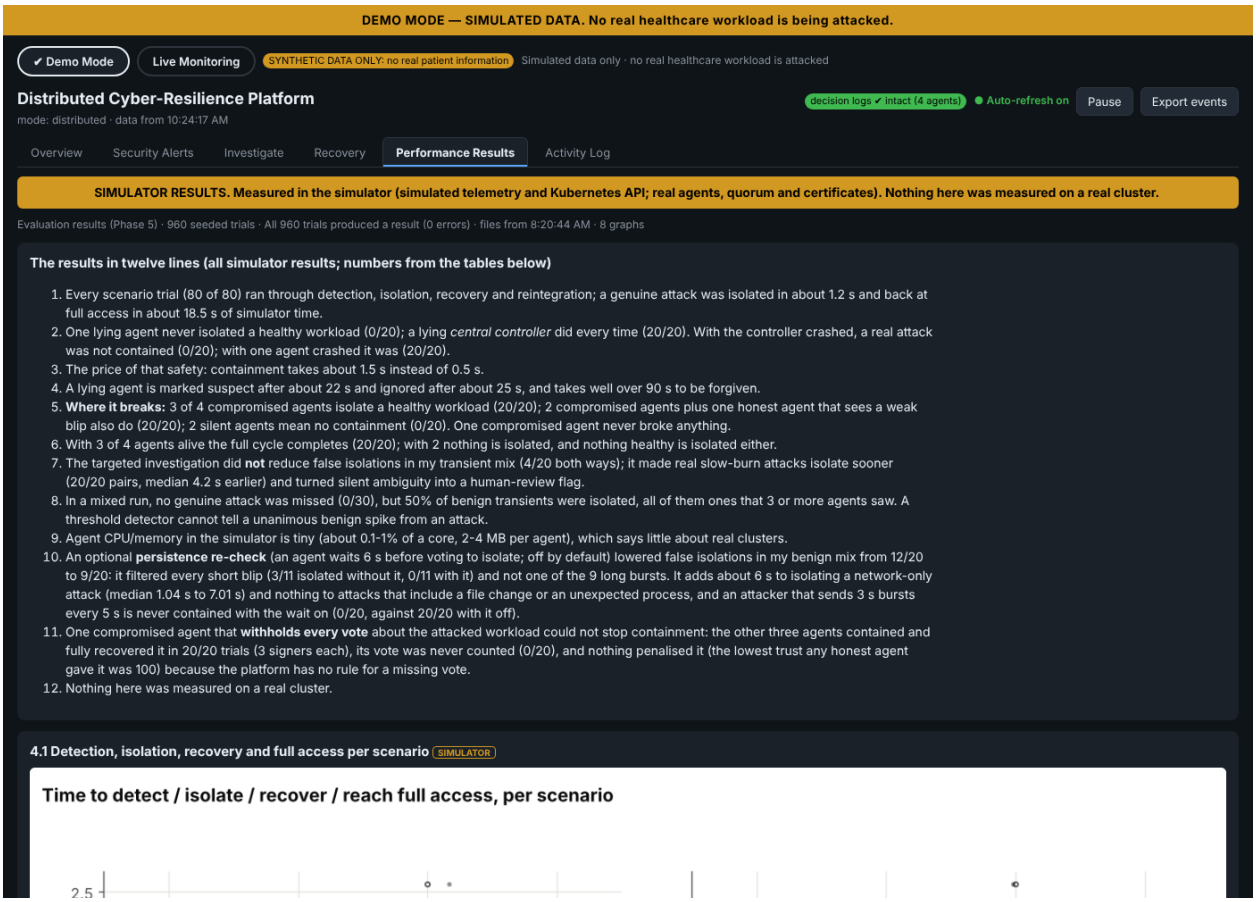
Overview in Demo Mode: counters, the four workload cards, and the Services table (what each service calls and what stops working if it is isolated).

You see	It means
Workload trust (0-100)	how much the agents trust this workload right now; falls while something looks wrong, restarts at 0 on isolation, must climb back before each step of access is restored
Agent trust (0-100)	how much the other agents trust this agent's evidence; falls only if its claims are contradicted
Evidence confidence (0 to 1)	how sure one agent is about one kind of signal on one workload; not a trust score
Recovery Check	pass or fail, not a score
Agreement Reached (3 of 4)	at least three agents signed; the only way anything is isolated

These are four separate things on purpose: there is no single combined 'security score'. Each agent is shown in exactly one of five states: **Agrees, Disagrees, Unavailable, Not yet responded, Message failed verification**. Only a counted, signed vote is 'Agrees'.



A **Security Alert** (the old 'incident' view): what triggered it, grouped evidence per agent and signal, every action as proposed, agreed, attempted and verified, and who agreed.



Performance Results: the graphs and tables stay visible; long explanations are collapsed. Labelled as simulator results.

5. Results measured in the simulator

These are SIMULATOR results. The telemetry and the Kubernetes API are simulated; the agents, signatures, 3-of-4 agreement, certificates and the recovery pipeline are the real code. Times such as '1.2 s' say nothing about real Kubernetes, where a pod takes tens of seconds to start. What carries over is *who* gets isolated, who does not, and what survives a failure. 20 trials per case unless stated; everything is reproducible with `python3 scripts/evaluate.py` (docs/RESULTS.md).

Question	What the simulator showed
Does the full cycle work?	80 of 80 scenario trials went through detection, isolation, recovery and return; a genuine attack was isolated in about 1.2 s and back at full access in about 18.5 s of simulator time.
One lying agent?	never isolated a healthy workload (0 of 20); the lying agent was marked suspect after about 22 s and ignored after about 25 s.
A lying or crashed central controller (the alternative design)?	a lying controller isolated a healthy workload 20 of 20 times; with the controller crashed a real attack was not contained (0 of 20); with one agent crashed it was (20 of 20).
Cost of the safety?	containment takes about 1.5 s instead of 0.5 s.
Where does it break?	3 of 4 compromised agents isolate a healthy workload (20 of 20); 2 compromised agents plus one honest agent seeing a weak blip also do (20 of 20); 2 silent agents mean nothing is isolated (0 of 20). One compromised agent never broke anything.
A compromised agent that refuses to vote ('obstruct')?	the other three contained and recovered the workload in 20 of 20 trials with exactly 3 signers; its vote was never counted (0 of 20). It was **not penalised**: the platform has no rule for a missing vote.
False alarms?	in a mixed run no genuine attack was missed (0 of 30) but 50% of benign traffic spikes were isolated, all of them seen by 3 or more agents: a threshold detector cannot tell a unanimous harmless spike from an attack.
Optional 6-second re-check (off by default)?	false isolations 12 of 20 down to 9 of 20: every short blip filtered (3 of 11 down to 0 of 11), none of the 9 long bursts. It delays a network-only attack from about 1.0 s to 7.0 s, and an attacker sending 3 s bursts every 5 s is **never** contained with it on (0 of 20, against 20 of 20 with it off).
Targeted investigation?	did **not** reduce false isolations in my mix (4 of 20 both ways); it made slow-burn attacks isolate about 4.2 s sooner and turned silent ambiguity into a human-review flag.

6. Results verified on the real cluster

LIVE-VERIFIED results: none yet. This table is a placeholder for the presenter to fill with their own runs (docs/LIVE_DEMO.md, section 9). Nothing in it has been verified. Do not describe a simulator result as a live result.

Test (runbook step)	Date	Result	Evidence (screenshot or log)
Pre-flight: verify-baseline, verify-isolation	not run		
Healthy state, 4 of 4 agents	not run		
Agent goes offline, 3 of 4 carry on, agent rejoins	not run		
Tripwire test: real detection, isolation, recovery, return	not run		
Compromised agent (false accusation)	not run		
Forgery rejected	not run		
Obstruct plus tripwire	not run		
Centralized comparison	not run		

7. A suggested demonstration (about 10 minutes)

In Demo Mode (safe, repeatable):

Scenario in the picker	What to say
Patient records being copied out of the EHR API	three of four agents agree, isolate it, a clean copy is checked, access returns in steps
A security agent falsely accuses the Login & Access Service	one liar cannot reach three votes; the liar is marked suspect
A security agent goes offline, then the EHR API is attacked	three agents are still enough
A compromised agent refuses to vote while the EHR API is attacked	the other three still act; the dashboard shows the refusing agent as 'Disagrees' with its reason

On the real cluster (only if you have run and verified it first): the healthy state, deleting one agent pod, and the **tripwire test**: a single harmless line appended to a designated, unused marker file in one workload, which the real agents detect and handle with the real pipeline. Exact commands, what to point at, and how to reset are in `docs/LIVE_DEMO.md`.

> **The one exception to 'attacks only in the simulator'.** The tripwire is not an attack: it is a harmless, operator-initiated change to one file that nothing reads, made with `python -c` (not a shell), refused for anything but the four workloads and for any other cluster. It triggers real isolation of a synthetic-data workload by design. It is documented as a controlled validation event with restore steps.

What the project deliberately does **not** contain: attack code, real patient data, or any way for the simulator's controls to reach a real cluster (the server refuses it, and tests prove it).

8. Limitations, stated plainly

- **Not verified on a real cluster.** Real pod start times, network-policy enforcement (Calico) and the admission webhook against a real API server are untested by the author.
- **Simulated telemetry.** Thresholds were tuned against invented signals; real traffic will differ, and the false-alarm rates depend on my choice of benign mix.
- **Telemetry is reported by the workload itself.** A fully compromised workload could lie about itself, and all four agents read the same source.
- **Three or more compromised agents** can isolate a healthy workload; two silent or obstructing agents stop containment. The design tolerates one bad agent, not two.
- **A sustained harmless spike seen by everyone looks like an attack** to a threshold detector; the optional re-check helps with short spikes and can be evaded by short bursts.
- **A missing vote costs the agent nothing** (no trust rule for it).
- **Agents keep history in temporary storage:** a deleted agent pod rejoins with empty history.
- **Four workloads only**, one per agent. Not a compliance claim: the HIPAA Security Rule categories in docs/HEALTHCARE.md are a mapping for discussion, with gaps listed.

9. Questions the faculty may ask, with honest answers

Q. Why four agents and three of four?

Four agents can tolerate one that is wrong or down and still reach three. One bad agent can neither cause an isolation (it cannot get three votes) nor block one (the other three suffice). It is a deliberate, small choice; docs/RESULTS.md shows exactly where it stops working.

Q. What if two agents are compromised?

Then it breaks, and the results say so: two compromised agents plus one honest agent that sees a weak blip can isolate a healthy workload, and two silent agents mean nothing is isolated. The claim is tolerance of one failure, not two.

Q. Has this been tested on a real cluster?

Not yet by me. All numbers are simulator results (real agents and cryptography, simulated telemetry and Kubernetes). A runbook and a harmless live test exist, and the live-verified table in this guide is empty until I run them.

Q. Why not one smart central controller?

It is a single point of failure and of compromise. In my simulation a lying controller isolated a healthy service 20 of 20 times and a crashed one never contained a real attack; the distributed design did neither. The price is slower containment (about 1.5 s against 0.5 s in the simulator).

Q. How does it recognise an attack?

Four signals measured by every agent independently: unexpected connections or traffic, unexpected programs, files that differ from the known-good copy, and bursts of failed logins. Weak or disputed evidence triggers a short investigation first.

Q. What about false alarms?

They exist. In my stress mix 50% of harmless spikes that three or more agents saw were isolated. An optional 6-second re-check lowers that (12 of 20 to 9 of 20) but delays some real attacks by about 6 s and can be evaded by short bursts, so it is off by default.

Q. What does isolating a workload do to patients and staff?

It cuts the workload off the network, so services that depend on it stop working: for example isolating the EHR API means the Patient Portal cannot show records. The dashboard shows this impact in advance. It is deliberate: a compromised system must not keep running.

Q. How do you know a recovered workload is clean?

A Recovery Check: healthy, files match the known-good copy, behaves normally, and it is a new instance. If it fails it is retried a limited number of times, then marked 'needs human attention' and stays isolated; it is never shown as healthy.

Q. Can a compromised agent forge another agent's messages?

Not undetected: messages are signed and agents use mutual TLS, so forged ones are rejected and the sender loses trust. A compromised agent can still lie with its own genuine identity; that is what the 3-of-4 rule handles.

Q. Is this HIPAA compliant?

No claim is made. docs/HEALTHCARE.md maps what the platform does to HIPAA Security Rule categories (access control, audit, integrity, contingency planning, transmission security) and lists the gaps; compliance needs much more than this project.

Q. Is any patient data real?

No. Every record is synthetic and labelled so on every page and in the application's responses.

Q. Is the 'tripwire' an attack?

No. It is one harmless line appended to an unused marker file, run by the operator on purpose, so the real agents have something genuine to detect. It is a narrow, documented exception, with restore steps.

Q. What would you do next?

Measure on a real cluster, tune thresholds on real traffic, add more than four workloads, add stronger sensors, and evaluate with realistic (still synthetic) workloads. These are future work, not done.

10. Glossary

Term	Plain meaning
Workload	one application service running in the cloud (portal, login, records API, database)
Agent	a small security program next to a workload; four identical peers
Evidence	a signed report from one agent about what it sees
Agreement Reached (3 of 4)	at least three agents signed a vote; also called a quorum
Isolation	cutting a workload off the network (reversible)
Recovery Check	tests that a replacement is healthy, matches the known-good copy and behaves normally
Normal Access Restored	the last step of the staged return, after limited, monitored and peer-confirmed access
Simulator	the test world with simulated telemetry and Kubernetes, used for all measured results
Synthetic data	made-up patient records; no real person is represented

Where to find things: `docs/RESULTS.md` (all numbers and graphs), `docs/HEALTHCARE.md` (requirements and safeguards), `docs/DASHBOARD.md`, `docs/LIVE_DEMO.md` (runbook), `docs/PROGRESS.md` (what was done, tests, limits).